Introduction to Data Science

Naeemullah Khan

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

LMH
Lady Margaret Hall
University of Oxford

KAUST Academy
King Abdullah University of Science and Technology
Stage 1 · Course 4

June 21, 2026

Module 1

NUMERICAL COMPUTING WITH NUMPY

Arrays, vectorized operations, and linear algebra

Module Content

This module covers building and manipulating arrays efficiently for numerical and scientific computing.

- **Why NumPy?**: fast fixed-type arrays vs. Python lists.
- **Creating arrays**: from lists, ranges, zeros, ones, and random values.
- **Attributes and dtypes**: shape, size, and type system.
- **Indexing and slicing**: 1-D, 2-D, and N-D access patterns.
- **Boolean and fancy indexing**: filter and select with conditions.
- **Broadcasting**: element-wise operations across different shapes.
- **Ufuncs and aggregations**: vectorized math and reductions.
- **Reshaping and combining**: `reshape`, `stack`, `concatenate`.
- **Linear algebra and random numbers**: `np.linalg` and `np.random`.
- **Saving and loading**: `.npy`, `.npz`, and performance tips.

Why NumPy?

Python lists are flexible but slow for numerical work. **NumPy** provides a fixed-type, fixed-size array (`ndarray`) and runs operations in optimized C code. It is the foundation of the entire scientific Python ecosystem.

```
import numpy as np
import time

# Pure Python loop
xs = list(range(1_000_000))
t0 = time.perf_counter()
ys = [x * 2 for x in xs]
print("loop:", time.perf_counter() - t0)        # ~ 0.05 s

# NumPy vectorized
arr = np.arange(1_000_000)
t0 = time.perf_counter()
arr2 = arr * 2
print("numpy:", time.perf_counter() - t0)       # ~ 0.001 s
```

Key idea

NumPy is fast because it stores data in **contiguous memory** with a single dtype, and operates on whole arrays at once instead of element by element in Python.

Creating Arrays

NumPy offers many ways to build arrays. Pick the constructor that best expresses your intent.

```
import numpy as np

np.array([1, 2, 3])             # from a Python list
np.zeros((3, 4))                # 3x4 array of zeros
np.ones((2, 5))                 # 2x5 array of ones
np.full((2, 2), 7)              # filled with a constant
np.eye(4)                       # 4x4 identity matrix

np.arange(0, 10, 2)             # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)            # 5 evenly spaced values: [0, 0.25, 0.5, 0.75, 1]
```

arange vs. linspace

`arange(start, stop, step)` controls the **step**. `linspace(start, stop, n)` controls the **number of points**. Use `linspace` when you need a precise count (e.g. for plotting).

Array Attributes and dtypes

Every `ndarray` carries metadata that describes its shape and storage.

```
a = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float32)

a.shape         # (2, 3)        -> rows, cols
a.ndim          # 2             -> number of dimensions
a.size          # 6             -> total elements
a.dtype         # float32        -> element type
a.itemsize      # 4             -> bytes per element
a.nbytes        # 24            -> total bytes
```

dtype	When to use
`float64`	Default for floats. Maximum precision.
`float32`	Half the memory. Standard on **GPUs** and most ML frameworks.
`int32` / `int64`	Integers. `int64` default on 64-bit systems.
`bool`	1 byte each. Used by boolean indexing.

Indexing and Slicing

NumPy uses comma-separated indexing for multi-dimensional arrays. Slicing returns a **view**, not a copy.

```
a = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]])

a[0, 0]            # 1            -> single element
a[1]               # [5, 6, 7, 8] -> entire row
a[:, 2]            # [3, 7, 11]   -> entire column
a[0:2, 1:3]        # [[2, 3], [6, 7]]   -> submatrix

a[1, 2] = 99       # mutates the original

# 3D arrays follow the same pattern
cube = np.zeros((4, 5, 6))     # shape (depth, rows, cols)
cube[0, :, :].shape            # (5, 6)
```

Slice vs. copy

`a[0:2, 1:3]` is a **view** into the original. To get an independent copy, use `a[0:2, 1:3].copy()`.

Boolean and Fancy Indexing

Index by a boolean mask (filter) or by a list of integer positions (gather).

```
a = np.array([10, 20, 30, 40, 50])

# Boolean indexing: filter by condition
mask = a > 25
print(mask)              # [False False  True  True  True]
print(a[mask])           # [30 40 50]
print(a[a > 25])         # same thing in one line

a[a > 25] = 0            # set matching elements

# Fancy indexing: pick by integer positions
idx = [0, 2, 4]
print(a[idx])            # [10 30 50]

# Combined conditions: use & and | (NOT and / or)
print(a[(a > 10) & (a < 50)])
```

Why this matters

Boolean masks are the NumPy / Pandas way to filter data. They replace explicit `for` loops in almost every data-cleaning task.

Element-wise Operations and Broadcasting

Arithmetic on arrays is **element-wise**. When shapes differ, NumPy uses **broadcasting** to align them.

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

a + b          # [11, 22, 33]   element-wise
a * b          # [10, 40, 90]
a ** 2         # [1, 4, 9]
a + 100        # [101, 102, 103]    scalar broadcasts

# 2D broadcasting
M = np.ones((3, 4))      # shape (3, 4)
v = np.array([1, 2, 3, 4])
print(M + v)             # v is broadcast across each row
```

Broadcasting rules

Two shapes are compatible when, comparing dimensions **from the right**: each pair is either equal, or one of them is 1. The size-1 dimension is then **stretched** to match the other.

Example: `(3, 4)` with `(4,)` aligns as `(3, 4)` with `(1, 4)` $\Rightarrow$ compatible.

Universal Functions and Aggregations

Universal functions (ufuncs) apply element-wise. **Aggregations** reduce an array along an axis.

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

# Ufuncs (element-wise)
np.sqrt(a)
np.exp(a)
np.log(a)
np.maximum(a, 3)              # element-wise max with a scalar

# Aggregations (reduce)
a.sum()           # 21         -> across all elements
a.mean()          # 3.5
a.std()           # standard deviation
a.min(),  a.max()
a.argmin(),  a.argmax()       # index of min / max

# Axis parameter
a.sum(axis=0)     # [5, 7, 9]    -> sum down columns
a.sum(axis=1)     # [6, 15]      -> sum across rows
a.cumsum(axis=1)  # cumulative
```

Reshaping and Combining Arrays

```
a = np.arange(12)

a.reshape(3, 4)            # 3 rows, 4 cols
a.reshape(3, -1)           # -1 means "infer this dimension"
a.ravel()                  # flatten -> view (when possible)
a.flatten()                # flatten -> always a copy

M = np.array([[1, 2], [3, 4]])
M.T                        # transpose

# Combining
np.concatenate([a, a], axis=0)
np.stack([a, a])           # adds a NEW axis
np.vstack([M, M])          # vertical stack  -> rows
np.hstack([M, M])          # horizontal stack -> cols
np.split(a, 3)             # [array1, array2, array3]
```

When shapes do not match

Most reshape errors come from forgetting `axis`, or using a tuple shape that does not multiply to the original size. Always check `a.shape` before and after.

Linear Algebra and Random Numbers

NumPy ships with `np.linalg` for linear algebra and `np.random` for sampling. Both are essential in machine learning.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

A @ B                # matrix product (preferred syntax)
np.matmul(A, B)      # same thing
np.linalg.inv(A)     # inverse
np.linalg.det(A)     # determinant
np.linalg.eig(A)     # eigenvalues and eigenvectors
np.linalg.norm(A)    # Frobenius norm

# Random sampling
rng = np.random.default_rng(seed=42)        # modern API
rng.normal(0, 1, size=(3, 4))               # standard normal
rng.uniform(0, 10, size=5)
rng.integers(0, 100, size=10)
rng.choice([1, 2, 3], size=5, replace=True)
```

Reproducibility

Always set a `seed` when you want experiments to be reproducible. The new `default_rng()` is preferred over the older `np.random.seed()`.

Saving, Loading, and Performance

Persistence

```
np.save("data.npy", arr)          # binary, fast
arr = np.load("data.npy")

np.savetxt("data.csv", arr,       # human-readable
           delimiter=",")
arr = np.loadtxt("data.csv",
                 delimiter=",")
```

Performance tips

- Vectorize: replace Python loops with array operations.
- Pick the right **dtype**: `float32` halves memory.
- Memory layout: **C-order** (row major, default) vs. **F-order** (column major).
- Use `np.where`, `np.select` for branching.

Mental shift

In NumPy you describe **what** you want done to the whole array. The library figures out the fast way to do it. Writing explicit element-by-element loops almost always means a slower program.

Module 2
TABULAR DATA WITH PANDAS

Series, DataFrames, cleaning, grouping, and merging

Module Content

This module covers loading, cleaning, reshaping, and analysing structured datasets using Pandas Series and DataFrames.

- **Series**: labeled 1-D arrays built on NumPy.
- **DataFrames**: tabular data with labeled rows and columns.
- **Indexing**: `.loc` vs. `.iloc` for label and positional access.
- **Inspecting data**: `info()`, `describe()`, and `head()`.
- **Missing data**: detecting, dropping, and filling `NaN`.
- **Type conversion, strings, and sorting**: cleaning real-world data.
- **GroupBy and aggregation**: split-apply-combine patterns.
- **Merging and joining**: combining DataFrames like SQL joins.
- **Apply, map, and method chaining**: expressive transformations.
- **Time series and categoricals**: working with dates and ordered data.

From NumPy to Pandas: the Series

A **Series** is a one-dimensional NumPy array with a labeled index. Think of it as a labeled vector.

```
import pandas as pd

s = pd.Series([10, 20, 30, 40], index=["a", "b", "c", "d"])
print(s)
# a    10
# b    20
# c    30
# d    40
# dtype: int64

s["b"]            # 20    -> label-based access
s[s > 15]         # boolean filter (just like NumPy)
s.mean()          # 25.0
```

Index alignment

When you combine two Series, Pandas **aligns by index label**, not by position. Missing labels become `NaN`. This is the single biggest difference from a plain list.

The DataFrame: Tabular Data

A **DataFrame** is a 2-D table where every column is a Series. Columns can have different dtypes.

```
df = pd.DataFrame({
    "name":  ["Aisha", "Omar", "Lina", "Tariq"],
    "age":   [22, 25, 23, 28],
    "score": [95.0, 88.5, 91.0, 76.5],
})

# Read from common sources
pd.read_csv("data.csv")
pd.read_json("data.json")
pd.read_excel("data.xlsx")
pd.DataFrame(np_array, columns=["x", "y", "z"])
```

Property	Meaning
`df.columns`	column labels
`df.index`	row labels
`df.shape`	`(rows, cols)`
`df["age"]`	a single column (returns a Series)
`df[["age","score"]]`	multiple columns (returns a DataFrame)

Indexing: .loc vs. .iloc

Pandas offers two complementary indexers. Choose by what you have: a **label** or a **position**.

```
df.loc[0, "name"]              # by label  -> "Aisha"
df.loc[0:2, ["name", "age"]]   # rows 0..2, two columns
df.loc[df["age"] > 24]         # boolean filter on rows

df.iloc[0, 0]                  # by position -> "Aisha"
df.iloc[0:2, :]                # first two rows, all columns
df.iloc[-1]                    # last row

# Setting values follows the same rules
df.loc[df["score"] < 80, "score"] = 80   # cap low scores
```

Rule of thumb

Use `.loc` for **labels and conditions**. Use `.iloc` for **integer positions**. Mixing them is the source of many beginner bugs.

Inspecting Data

Before doing anything else, look at your data. Pandas has standard methods for this.

```
df.head(5)              # first 5 rows
df.tail(5)              # last 5 rows
df.sample(5)            # random sample

df.info()               # dtypes + non-null counts + memory
df.describe()           # numeric summary: mean, std, min, max, ...
df.describe(include="object")   # for string columns

df.shape                # (rows, cols)
df.dtypes               # column types
df["category"].value_counts()   # frequency table
df["category"].nunique()        # number of unique values
```

First-look checklist

- How big is the table? (`shape`)
- What types? (`info`)
- Any missing values? (`info` or `isna().sum()`)
- How are values distributed? (`describe`, `value_counts`)

Handling Missing Data

Real data has gaps. Pandas represents them with `NaN` (and `NaT` for missing dates).

```
df.isna().sum()                  # count missing per column
df.notna()                       # opposite mask

# Drop rows or columns with any missing values
df.dropna()                      # row with any NaN
df.dropna(subset=["age"])        # only consider 'age'
df.dropna(axis=1, thresh=100)    # cols with >=100 non-null

# Fill missing values
df["age"] = df["age"].fillna(df["age"].median())
df["city"] = df["city"].fillna("unknown")
df = df.fillna(method="ffill")  # forward-fill (time series)
df["x"] = df["x"].interpolate() # numeric interpolation
```

Imputation choices

Replace missing numbers with the **median** (robust to outliers). Replace missing categories with "unknown" or the **mode**. For time series, **forward-fill** or **interpolate**.

Type Conversion, Strings, and Sorting

```python
# Type conversion
df["age"] = df["age"].astype("int32")
df["price"] = pd.to_numeric(df["price"], errors="coerce")  # bad -> NaN
df["date"]  = pd.to_datetime(df["date"])

# String operations via the .str accessor
df["name"].str.lower()
df["email"].str.contains("@example.com")
df["name"].str.split().str[0]      # take first token

# Sorting
df.sort_values("score", ascending=False)
df.sort_values(["dept", "score"], ascending=[True, False])
df.nlargest(5, "score")            # top-5 by score
df.nsmallest(5, "age")
```

Why convert types early

A column read as `object` (string) cannot be summed or compared numerically. Fix dtypes right after loading the data, then everything downstream is faster and safer.

GroupBy and Aggregation

groupby is the workhorse of tabular analysis: split the table by a key, apply a function, combine the results.

```
df.groupby("dept")["score"].mean()

# Multiple aggregations at once
df.groupby("dept").agg(
    avg_score=("score", "mean"),
    max_score=("score", "max"),
    n_students=("score", "count"),
)

# Group by multiple keys
df.groupby(["dept", "year"])["score"].mean()

# Transform: same shape as the original (e.g. z-scores within group)
df["z"] = df.groupby("dept")["score"].transform(
    lambda s: (s - s.mean()) / s.std()
)

# Pivot tables for cross-tabulation
pd.pivot_table(df, values="score",
               index="dept", columns="year", aggfunc="mean")
```

Merging and Joining

`merge` combines two DataFrames on shared keys. `concat` stacks them.

```
students = pd.DataFrame({"id": [1, 2, 3], "name": ["A", "B", "C"]})
grades   = pd.DataFrame({"id": [1, 2, 4], "grade": [95, 88, 70]})

pd.merge(students, grades, on="id", how="inner")    # only matches
pd.merge(students, grades, on="id", how="left")     # keep all students
pd.merge(students, grades, on="id", how="outer")    # keep everything

# Stacking
pd.concat([df1, df2], axis=0)          # rows on top of rows
pd.concat([df1, df2], axis=1)          # columns side by side
```

how=	Result
inner	only rows present in both tables
left	every row in the **left** table; `NaN` where right is missing
right	mirror of left
outer	every row in either table

Apply, Map, and Method Chaining

```python
# Element-wise on a Series
df["score"].map(lambda x: x / 100)

# Row-wise or column-wise on a DataFrame
df["full"] = df.apply(lambda r: f"{r['name']} ({r['age']})", axis=1)

# Method chaining: a clean preprocessing pipeline
clean = (
    pd.read_csv("raw.csv")
      .dropna(subset=["age"])
      .assign(age_group=lambda d: pd.cut(d["age"], [0, 18, 35, 65, 99]))
      .query("score >= 60")
      .sort_values("score", ascending=False)
      .reset_index(drop=True)
)
```

Performance reminder

`apply(axis=1)` runs row by row in Python and is **slow**. Prefer vectorized expressions, `np.where`, or built-in Pandas methods whenever possible.

Time Series and Categoricals

```python
# A DatetimeIndex unlocks time-series operations
df = df.set_index(pd.to_datetime(df["date"]))

df.resample("M").mean()        # monthly averages
df["price"].rolling(7).mean()  # 7-day rolling mean
df["price"].shift(1)           # previous value
df["price"].diff()             # period-over-period change

# Categorical dtype: huge memory savings + correct ordering
df["dept"] = df["dept"].astype("category")
df["dept"].cat.categories
df["size"] = pd.Categorical(df["size"],
    categories=["S", "M", "L"], ordered=True)
```

When to use categoricals

Use `category` dtype when a string column has **few unique values** repeated many times (department, country, status). Memory drops dramatically and group-by becomes faster.

Module 3
DATA FORMATS — JSON & APIs

JSON structure, parsing, files, and real-world APIs

Module Content

This module covers reading, writing, and processing JSON — the most common format for web data and real-world APIs.

- **JSON structure**: mapping JSON types to Python equivalents.
- **Reading and writing strings**: `json.loads()` and `json.dumps()`.
- **Reading and writing files**: `json.load()` and `json.dump()`.
- **Custom encoders and decoders**: handling non-standard Python types.
- **Real-world API responses**: parsing and extracting data from live APIs.

JSON Structure and Python Equivalents

JSON (JavaScript Object Notation) is the de-facto data format of the web. It maps almost directly onto Python types.

JSON	Python	Example
object	`dict`	`{"name": "Ada"}`
array	`list`	`[1, 2, 3]`
string	`str`	`"hello"`
number (int/float)	`int / float`	`42, 3.14`
`true / false`	`True / False`	boolean
`null`	`None`	missing value

What JSON does *not* have

JSON has no dates, no tuples, no sets, no `NaN`, and no comments. Dates are usually transmitted as **ISO-8601 strings** ("2026-05-09") and parsed afterwards.

Reading and Writing JSON Strings

The `json` module converts between JSON text and Python objects.

```python
import json

# JSON text -> Python object
text = '{"name": "Ada", "skills": ["python", "numpy"], "age": 30}'
data = json.loads(text)
print(data["skills"][0])     # python

# Python object -> JSON text
out = json.dumps(data, indent=2, ensure_ascii=False)
print(out)
# {
#   "name": "Ada",
#   "skills": ["python", "numpy"],
#   "age": 30
# }
```

The "s" in loads / dumps

`loads` = "load string", `dumps` = "dump string". Both versions *without* the `s` (`json.load`, `json.dump`) work on **file objects**.

Reading and Writing JSON Files

```python
import json

# Write a Python object to a JSON file
record = {"user": "ada", "scores": [95, 88, 91]}

with open("record.json", "w", encoding="utf-8") as f:
    json.dump(record, f, indent=2)

# Read a JSON file back
with open("record.json", encoding="utf-8") as f:
    loaded = json.load(f)

print(loaded["scores"])     # [95, 88, 91]
```

Encoding and the indent argument

Pass `indent=2` for human-readable output. Pass `ensure_ascii=False` when your data contains non-ASCII characters (Arabic, Chinese, emoji) so they are saved as themselves rather than `\uXXXX` escapes.

Custom Encoders and Decoders

Some Python types are not JSON-native. Provide a custom encoder when serializing them.

```python
import json
from datetime import datetime
from decimal import Decimal

class MyEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat()           # 2026-05-09T11:30:00
        if isinstance(obj, Decimal):
            return float(obj)
        return super().default(obj)

data = {"event": "login", "when": datetime.now(), "amount": Decimal("12.50")}
print(json.dumps(data, cls=MyEncoder, indent=2))

# Decoding side: object_hook can rebuild custom types
def parse_dates(d):
    for k, v in d.items():
        if isinstance(v, str) and v.endswith("Z"):
            try: d[k] = datetime.fromisoformat(v.rstrip("Z"))
            except ValueError: pass
    return d

obj = json.loads(text, object_hook=parse_dates)
```

Real-world API Responses

APIs return nested JSON. The job is usually to drill into it and extract a flat table.

```
import requests
import pandas as pd

resp = requests.get("https://api.example.com/users")
payload = resp.json()        # already-parsed Python object

# Typical shape:
# {"status":"ok","data":{"users":[
#   {"id":1,"name":"Ada","city":{"name":"Dhahran"}}, ...]}}

users = payload["data"]["users"]                       # drill in
rows  = [{"id": u["id"], "name": u["name"],
          "city": u["city"]["name"]} for u in users]   # flatten

df = pd.DataFrame(rows)
# Or in one step:
df = pd.json_normalize(users, sep="_")
```

The pattern

Inspect the JSON, **drill in** to the relevant list, **flatten** each record, then build a DataFrame. `pd.json_normalize` automates the last two steps for many shapes.

Module 4
DATA VISUALIZATION

Matplotlib and Seaborn for charts and statistical plots

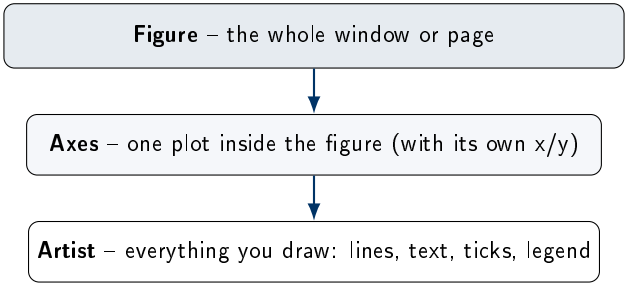
Module Content

This module covers communicating data insights effectively using Matplotlib and Seaborn.

- **Matplotlib architecture**: figures, axes, and the rendering pipeline.
- **Pyplot vs. OO interface**: when to use each approach.
- **Basic charts**: line, bar, scatter, and histogram.
- **Customization and subplots**: titles, labels, legends, and layouts.
- **Seaborn**: statistical plotting built on Matplotlib.
- **Categorical and relational plots**: `boxplot`, `violinplot`, `scatterplot`.
- **Heatmaps, pair plots, and styling**: correlation matrices and themes.

Matplotlib Architecture

Matplotlib is built on three layers. Understanding them removes most "which function should I call" confusion.



Mental model

A **Figure** is the canvas. It holds one or more **Axes** (each is one chart). Every line, label, and tick on those axes is an **Artist**. Almost every customization changes some Artist's properties.

Pyplot vs. the Object-Oriented Interface

Matplotlib offers two styles. They produce identical figures.

Pyplot (quick, MATLAB-like)

```python
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [4, 1, 5])
plt.title("Quick plot")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Object-oriented (recommended)

```python
fig, ax = plt.subplots()

ax.plot([1, 2, 3], [4, 1, 5])
ax.set_title("OO plot")
ax.set_xlabel("x")
ax.set_ylabel("y")
plt.show()
```

Which to use

Use `plt....` for quick exploration. Use **Figure** and **Axes** objects (`fig, ax = plt.subplots()`) for any plot you will customize, save, or reuse. The OO style scales to multi-panel figures cleanly.

Basic Charts

```python
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(0, 10, 50)
y = np.sin(x)

fig, ax = plt.subplots()

ax.plot(x, y, label="sin(x)")         # line plot
ax.scatter(x, y, s=20, c="red")       # scatter
ax.bar(["A", "B", "C"], [3, 7, 5])    # bar chart
ax.hist(np.random.randn(1000), bins=30)   # histogram
ax.pie([30, 45, 25], labels=["A", "B", "C"], autopct="%.0f%%")
```

Chart	Use it for
line	ordered or continuous data over an axis (e.g. time)
scatter	relationship between two numeric variables
bar	comparing categories
histogram	distribution of one numeric variable
pie	shares of a whole (use **sparingly**; bars are usually clearer)

Customization and Subplots

```
fig, axes = plt.subplots(2, 2, figsize=(8, 6))   # 2x2 grid

axes[0, 0].plot(x, np.sin(x), color="navy",
                linestyle="--", linewidth=2, marker="o")
axes[0, 0].set_title("Sine")
axes[0, 0].set(xlabel="x", ylabel="sin(x)")
axes[0, 0].legend(["sin"], loc="upper right")
axes[0, 0].grid(alpha=0.3)
axes[0, 1].scatter(x, np.cos(x), c=x, cmap="viridis")
axes[1, 0].bar(["A", "B"], [10, 20])
axes[1, 1].hist(np.random.randn(500), bins=20)

fig.suptitle("Four-panel figure")
fig.tight_layout()
fig.savefig("plot.png", dpi=300, bbox_inches="tight")
```

Saving figures

`savefig` accepts PNG, SVG, and PDF. Use `dpi=300` for print, `bbox_inches="tight"` to crop whitespace, and SVG/PDF for vector quality in reports and slides.

Seaborn: Statistical Plotting

Seaborn sits on top of Matplotlib and produces statistical charts directly from a DataFrame.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="whitegrid")        # clean default look

# Distribution plots
sns.histplot(data=df, x="age", bins=20, kde=True)
sns.kdeplot(data=df,  x="age", hue="dept", fill=True)
sns.ecdfplot(data=df, x="age")

plt.show()
```

Why prefer Seaborn?

Seaborn understands DataFrames natively. Pass `data=df`, name your columns by string, and it handles axes, legends, and statistical aesthetics for you. The result is shorter code and better defaults.

Categorical and Relational Plots

```
# Categorical (one axis is a category)
sns.boxplot(data=df,     x="dept", y="score")
sns.violinplot(data=df, x="dept", y="score", inner="quartile")
sns.stripplot(data=df,  x="dept", y="score", jitter=True)
sns.barplot(data=df,     x="dept", y="score", estimator="mean")
sns.countplot(data=df,  x="dept")

# Relational (both axes are numeric)
sns.scatterplot(data=df, x="age", y="score",
                hue="dept", size="hours", style="gender")
sns.lineplot(data=ts,    x="date", y="price", hue="ticker")
```

The hue / size / style trio

Most Seaborn functions accept `hue`, `size`, and `style`. They map a column to color, marker size, or marker shape, letting you encode **up to five dimensions** on a 2-D chart.

Heatmaps, Pair Plots, and Styling

```
# Correlation heatmap
corr = df.select_dtypes("number").corr()
sns.heatmap(corr, annot=True, cmap="coolwarm",
            vmin=-1, vmax=1, square=True)

# Pair plot: every numeric pair as a scatter, diagonals as histograms
sns.pairplot(df, hue="dept", diag_kind="kde")

# Theming
sns.set_theme(style="whitegrid", palette="deep", font_scale=1.1)
plt.rcParams["figure.dpi"] = 120
```

Function level	Returns
Axes-level (`histplot`, `boxplot`, `scatterplot`)	a single **Axes**; you control the figure
Figure-level (`pairplot`, `relplot`, `catplot`)	a multi-panel **Figure** (`FacetGrid`)

Module 5

EXPLORATORY DATA ANALYSIS (EDA)

From raw data to insight: quality, distributions, and patterns

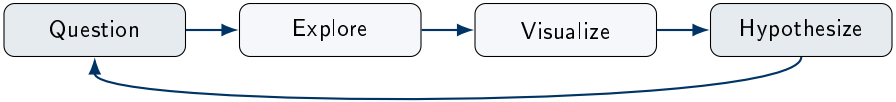
Module Content

This module covers a systematic process to understand any dataset — from quality checks to statistical profiling and machine learning readiness.

- **EDA as a process**: the question-explore-visualize-hypothesize loop.
- **Data quality assessment**: missing values, duplicates, and anomalies.
- **Univariate analysis**: distributions, histograms, and summary statistics.
- **Bivariate and multivariate analysis**: correlations and relationships.
- **Outlier detection**: IQR, z-scores, and visual methods.
- **ML readiness**: leakage, class imbalance, and feature scaling.
- **Worked example**: a full EDA skeleton on the Titanic dataset.

Exploratory Data Analysis as a Process

EDA is not a single command. It is a structured loop of asking questions and looking at the data until you understand it well enough to model it.



The deliverable

Good EDA produces a short report with three sections: **observations** (what we saw), **hypotheses** (what we now suspect), and **recommendations** (what to do next: clean, model, collect more).

Data Quality Assessment

Before any analysis, check four properties of the dataset.

Dimension	What to check
Completeness	How many missing values per column? `df.isna().mean()` as a fraction.
Consistency	Same units, same date format, same spelling for categories (`"USA"` vs `"U.S."`).
Accuracy	Plausible ranges (no negative ages, no temperatures of 999).
Timeliness	Is the data fresh enough for the question?

```
# Quick quality snapshot
df.isna().mean().sort_values(ascending=False)   # % missing per col
df.duplicated().sum()                           # duplicate rows
df["age"].between(0, 120).all()                 # range check
df["country"].value_counts()                    # spot odd categories
```

Univariate Analysis

Look at one column at a time. Characterize its **distribution**: center, spread, and shape.

```
df["age"].describe()                    # mean, std, quartiles
df["age"].skew()                        # > 0 long right tail
df["age"].kurt()                        # heavier tails than normal

import seaborn as sns
sns.histplot(df["age"], bins=30, kde=True)
sns.boxplot(x=df["age"])                # see outliers and IQR
```

Statistic	Interpretation
Mean / Median	center of the distribution; large gap suggests skew or outliers
Std / IQR	spread; IQR is robust to outliers
Skewness	asymmetry: 0 is symmetric, >0 right-skewed, <0 left-skewed
Kurtosis	tail weight: >0 heavier tails, more outliers than a normal

```python
# Two numeric columns: correlation and scatter
df[["age", "score"]].corr()
sns.scatterplot(data=df, x="age", y="score")

# Numeric vs.\ category: grouped statistics and box plots
df.groupby("dept")["score"].agg(["mean", "std", "count"])
sns.boxplot(data=df, x="dept", y="score")

# Two categories: cross-tabulation
pd.crosstab(df["dept"], df["pass"], normalize="index")

# All numeric columns at once
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
sns.pairplot(df, hue="dept")
```

Correlation is not causation

A strong correlation tells you two columns **move together**. It does not tell you whether one *causes* the other, or whether a third variable drives both. Always pair correlations with domain reasoning.

Outlier Detection

Outliers are values far from the rest of the data. Detect them with simple statistical rules and confirm visually.

```
# IQR (interquartile range) method - robust, non-parametric
Q1, Q3 = df["price"].quantile([0.25, 0.75])
IQR = Q3 - Q1
mask = (df["price"] < Q1 - 1.5 * IQR) | (df["price"] > Q3 + 1.5 * IQR)
outliers = df[mask]

# Z-score method - assumes roughly normal distribution
from scipy import stats
z = np.abs(stats.zscore(df["price"]))
outliers = df[z > 3]

# Visual check
sns.boxplot(x=df["price"])
```

Decide before you delete

Some outliers are **errors** (impossible values, typos) and should be removed or corrected. Others are **rare but real** (a billionaire in a salary dataset) and may be the most informative points. Always investigate before dropping.

Concerns Specific to Machine Learning

EDA for ML projects has two extra concerns that easy to miss.

Target leakage

- A feature that contains future or target-derived information.
- Example: predicting churn using `cancellation_date`.
- Symptom: a single feature is suspiciously predictive.
- Fix: remove or rebuild the feature using only data available at prediction time.

Class imbalance

- One class dominates the target.
- Example: 99% legitimate, 1% fraud.
- Symptom: high accuracy but useless predictions.
- Fix: report **precision/recall/F1**, resample, or use class weights.

Catch these in EDA

Both problems are visible in basic EDA. `value_counts()` on the target reveals imbalance. A correlation matrix that has one feature near ± 1 with the target screams *leakage*.

A Worked EDA Skeleton: Titanic

A typical first-pass EDA on a tabular dataset.

```
import pandas as pd, seaborn as sns, matplotlib.pyplot as plt

df = pd.read_csv("titanic.csv")

# 1. SHAPE & TYPES
df.shape, df.dtypes, df.head()

# 2. QUALITY
df.isna().mean().sort_values(ascending=False)
df.duplicated().sum()

# 3. UNIVARIATE
df["Age"].describe()
sns.histplot(df["Age"].dropna(), bins=30, kde=True)
df["Survived"].value_counts(normalize=True)        # class balance

# 4. BIVARIATE / TARGET
df.groupby("Sex")["Survived"].mean()
df.groupby("Pclass")["Survived"].mean()
sns.heatmap(df.corr(numeric_only=True), annot=True)

# 5. OUTPUT a short report:
#    observations -> hypotheses -> next steps
```

Thank you!

Questions and discussion

Course takeaway

Data science is the discipline of turning raw data into reliable answers. The tools change; the workflow — inspect, clean, visualize, hypothesize — does not.